Prompty EN dla modulu, pakietu, wersje edu, safe, aggresive

Radosław — zrobię to **dokładnie tak, jak chcesz**:
dla **każdej z 10 grup refaktoryzacji** przygotuję **3 osobne wersje promptów**:
1. **Edukacyjna** — AI nie zmienia kodu, tylko uczy, tłumaczy, wskazuje błędy, daje przykłady.
2. **Ultra-bezpieczna** — AI zmienia tylko to, co 100% pewne, minimalny diff, zero ryzyka.
3. **Agresywna** — AI robi maksymalnie dużo, globalnie, hurtowo, ale nadal bez zmiany logiki.
I to wszystko **dla refaktoryzacji modułów/pakietów**, czyli nie całego repo, ale większego fragmentu kodu.
To będzie **30 gotowych promptów** (3×10), idealnych do Twojego kursu i praktyki.

⭐ GRUPA 1 — Naming (technical, non-domain)

🎓 1A. Wersja edukacyjna (module/package)

PROMPT:
Analyze the naming conventions in this module/package.
Do NOT modify the code.
Instead:
• identify unclear, abbreviated, or generic names,
• explain why they are problematic,
• propose improved technical names (non-domain),
• show before/after examples,
• teach the underlying naming principles.

Return a structured educational review.

🛡️ 1B. Wersja ultra-bezpieczna (module/package)

PROMPT:
Refactor naming in this module/package using only 100% safe, technical transformations:
◇ expand abbreviations (usr → user, cfg → configuration),
◇ improve boolean names (active → isActive),
◇ replace generic method names with technical ones (check → isValid),
◇ improve variable names based on type and usage.

Do NOT rename anything requiring domain knowledge.
Do NOT change public APIs unless guaranteed safe.
Return the refactored code + list of changes.

🔥 1C. Wersja agresywna (module/package)

PROMPT:
Perform an aggressive naming refactor across this module/package:
◇ expand all abbreviations,
◇ rename all generic methods to precise technical names,

◇ rename variables for clarity,
◇ unify naming conventions across all files,
◇ rename helper/manager/utils classes to meaningful technical names,
◇ apply consistent boolean naming,
◇ refactor internal APIs if needed (non-public).

Do NOT change business logic.
Return full refactored code + global rename map.

⭐ GRUPA 2 — Logic simplification

🎓 2A. Edukacyjna

PROMPT:
Analyze logic complexity in this module/package.
Do NOT modify code.
Explain:
◇ where guard clauses should be used,
◇ where boolean extraction improves readability,
◇ where negations should be eliminated,
◇ where boolean flags are harmful,
◇ provide before/after examples.

Return an educational explanation.

🛡️ 2B. Ultra-bezpieczna

PROMPT:
Simplify logic in this module/package using only safe transformations:
◇ convert nested if-statements to guard clauses,
◇ extract complex boolean expressions into named variables,
◇ eliminate negations when unambiguous,
◇ simplify boolean comparisons.

Do NOT modify method signatures.
Do NOT change logic.
Return refactored code + list of changes.

🔥 2C. Agresywna

PROMPT:
Perform an aggressive logic simplification across this module/package:
◇ rewrite all nested conditionals into guard clauses,
◇ extract all multi-condition expressions,
◇ remove all boolean flags (split methods),
◇ simplify all boolean expressions,

◇ unify logical structure across files.

Do NOT change business logic.
Return full refactored code + summary.

⭐ GRUPA 3 — Code structure

🎓 3A. Edukacyjna

PROMPT:
Review the structural quality of this module/package.
Do NOT modify code.
Explain:

◇ where magic numbers appear,

◇ where dead code exists,

◇ where println should be replaced,

◇ where class structure is inconsistent,

◇ where anonymous classes should be lambdas.

Provide examples and reasoning.

🛡️ 3B. Ultra-bezpieczna

PROMPT:
Apply safe structural refactoring:

◇ extract magic numbers used multiple times,

◇ remove unused imports and commented code,

◇ replace println with logger,

◇ reorder class elements safely,

◇ convert anonymous classes to lambdas when trivial.

Do NOT modify public APIs.
Return refactored code + list of changes.

🔥 3C. Agresywna

PROMPT:
Perform aggressive structural cleanup:

◇ extract all magic numbers,

◇ remove all dead code,

◇ enforce consistent class structure,

◇ convert all anonymous classes to lambdas,

◇ convert lambdas to method references,

◇ unify logger usage.

Return full refactored code + structural diff.

⭐ GRUPA 4 — Null safety

🎓 4A. Edukacyjna

PROMPT:

Analyze null-safety issues in this module/package.
Do NOT modify code.
Explain:

◇ where null checks are missing,

◇ where @NotNull/@Nullable should be used,

◇ where Optional is appropriate,

◇ where null returns are dangerous.

Provide examples and reasoning.

🛡️ 4B. Ultra-bezpieczna

PROMPT:

Apply safe null-safety improvements:

◇ add @NotNull/@Nullable based on usage,

◇ replace return null with empty collections,

◇ add Objects.requireNonNull() at boundaries.

Do NOT introduce Optional unless return type already uses it.
Return refactored code + list of changes.

🔥 4C. Agresywna

PROMPT:

Perform aggressive null-safety refactoring:

◇ annotate all parameters and returns,

◇ replace all null returns with safe alternatives,

◇ convert null-handling to Optional where appropriate,

◇ enforce fail-fast validation everywhere.

Return full refactored code + summary.

⭐ GRUPA 5 — Immutability

🎓 5A. Edukacyjna

PROMPT:

Explain immutability issues in this module/package.
Do NOT modify code.
Identify:

◇ missing final,

◇ mutable collections,

◇ unnecessary wrappers,
◇ mutable fields.

Provide examples and reasoning.

🛡️ 5B. Ultra-bezpieczna

PROMPT:
Apply safe immutability improvements:
◇ add final where variables are never reassigned,
◇ replace new ArrayList<>() with List.of() when list is never modified,
◇ replace wrappers with primitives when null is impossible.

Return refactored code + list of changes.

🔥 5C. Agresywna

PROMPT:
Perform aggressive immutability refactoring:
◇ enforce final everywhere,
◇ convert all immutable collections to List.of()/Map.of(),
◇ convert wrappers to primitives,
◇ mark fields final when safe.

Return full refactored code + summary.

⭐ GRUPA 6 — Formatting & style

🎓 6A. Edukacyjna

PROMPT:
Review formatting and style issues.
Do NOT modify code.
Explain:
◇ inconsistent indentation,
◇ missing blank lines,
◇ inconsistent logger usage,
◇ import issues.

Provide examples.

🛡️ 6B. Ultra-bezpieczna

PROMPT:
Apply safe formatting improvements:
◇ unify indentation,
◇ add blank lines between logical blocks,

◇ clean up imports,
◇ unify bracket style.

Return refactored code.

🔥 6C. Agresywna

PROMPT:
Apply aggressive style unification:
◇ enforce consistent formatting across all files,
◇ unify logger usage,
◇ unify naming of constants,
◇ enforce consistent method ordering.

Return full refactored code.

⭐ GRUPA 7 — Exceptions

🎓 7A. Edukacyjna

PROMPT:
Explain exception-handling issues.
Do NOT modify code.
Identify:
◇ empty catch blocks,
◇ overly broad catches,
◇ unnecessary try/catch,
◇ missing logging.

Provide examples.

🛡️ 7B. Ultra-bezpieczna

PROMPT:
Apply safe exception improvements:
◇ replace empty catch with logging,
◇ narrow catch types when obvious,
◇ remove unnecessary casts.

Return refactored code.

🔥 7C. Agresywna

PROMPT:
Perform aggressive exception refactoring:
◇ rewrite all empty catches,
◇ narrow all broad catches,

◇ remove redundant try/catch,
◇ unify exception-handling style.

Return full refactored code.

⭐ GRUPA 8 — Annotations & boilerplate

🎓 8A. Edukacyjna

PROMPT:
Explain annotation and boilerplate issues.
Do NOT modify code.
Identify:
◇ missing @Override,
◇ manual getters/setters,
◇ manual equals/hashCode,
◇ missing constructors.

Provide examples.

🛡️ 8B. Ultra-bezpieczna

PROMPT:
Apply safe annotation improvements:
◇ add @Override,
◇ replace trivial getters/setters with Lombok,
◇ replace trivial equals/hashCode with Lombok.

Return refactored code.

🔥 8C. Agresywna

PROMPT:
Perform aggressive boilerplate cleanup:
◇ convert all getters/setters to Lombok,
◇ convert all builders to @Builder,
◇ convert all equals/hashCode to Lombok,
◇ generate constructors.

Return full refactored code.

⭐ GRUPA 9 — Loops & collections

🎓 9A. Edukacyjna

PROMPT:
Explain loop and collection issues.

Do NOT modify code.
Identify:

◇ index loops,

◇ raw types,

◇ mutable collections,

◇ missing generics.

Provide examples.

🛡️ 9B. Ultra-bezpieczna

PROMPT:

Apply safe loop/collection improvements:

◇ replace index loops with for-each when index unused,

◇ add generics,

◇ add diamond operator.

Return refactored code.

🔥 9C. Agresywna

PROMPT:

Perform aggressive loop/collection refactoring:

◇ convert loops to streams,

◇ replace mutable collections with immutable ones,

◇ enforce generics everywhere.

Return full refactored code.

⭐ GRUPA 10 — Comments & documentation

🎓 10A. Edukacyjna

PROMPT:

Explain documentation issues.
Do NOT modify code.
Identify:

◇ redundant comments,

◇ commented-out code,

◇ unclear TODOs,

◇ missing Javadoc.

Provide examples.

🛡️ 10B. Ultra-bezpieczna

PROMPT:

Apply safe documentation cleanup:

◇ remove commented-out code,
◇ remove comments describing “what”,
◇ normalize TODO/FIXME.

Return refactored code.

🔥 10C. Agresywna

PROMPT:
Perform aggressive documentation refactoring:
◇ remove all redundant comments,
◇ rewrite unclear TODOs,
◇ improve Javadoc,
◇ enforce consistent documentation style.

Return full refactored code.

🔥 Chcesz teraz:

- jedną wielką tabelę porównawczą (30 promptów × 10 grup)?
- wersję PDF / kursową?
- wersję „ALL-IN-ONE” dla modułów?
- wersję „ALL-IN-ONE” dla całego repo?

Powiedz, co dalej.